2019 IEEE 39th International Conference on Distributed Computing Systems (ICDCS)

DÏoT: A Federated Self-learning Anomaly Detection System for IoT

Thien Duc Nguyen[1], Samuel Marchal[2], Markus Miettinen[1], Hossein Fereidooni[1], N. Asokan[2], and Ahmad-Reza Sadeghi[1]

[1]*TU Darmstadt, Germany - {ducthien.nguyen,markus.miettinen,hossein.fereidooni,ahmad.sadeghi}@trust.tu-darmstadt.de*
[2]*Aalto University, Finland - samuel.marchal@aalto.fi, asokan@acm.org*

***Abstract*—IoT devices are increasingly deployed in daily life. Many of these devices are, however, vulnerable due to insecure design, implementation, and configuration. As a result, many networks *already* have vulnerable IoT devices that are easy to compromise. This has led to a new category of malware specifically targeting IoT devices. However, existing intrusion detection techniques are not effective in detecting compromised IoT devices given the massive scale of the problem in terms of the number of different types of devices and manufacturers involved.**

In this paper, we present DÏoT, an autonomous self-learning distributed system for detecting compromised IoT devices. DÏoT builds effectively on device-type-specific communication profiles without human intervention nor labeled data that are subsequently used to detect anomalous deviations in devices' communication behavior, potentially caused by malicious adversaries. DÏoT utilizes a federated learning approach for aggregating behavior profiles efficiently. To the best of our knowledge, it is the first system to employ a federated learning approach to anomaly-detection-based intrusion detection. Consequently, DÏoT can cope with emerging new and unknown attacks. We systematically and extensively evaluated more than 30 off-the-shelf IoT devices over a long term and show that DÏoT is highly effective ($95.6\,\%$ detection rate) and fast ($\approx 257\,\mathrm{ms}$) at detecting devices compromised by, for instance, the infamous Mirai malware. DÏoT reported *no false alarms* when evaluated in a real-world smart home deployment setting.

***Index Terms*—Internet of Things; IoT security; IoT malware; anomaly detection; federated deep learning; self-learning**

I. Introduction

Many new device manufacturers are entering the IoT device market, bringing out products at an ever-increasing pace. This "rush-to-market" mentality of some manufacturers has led to poor product design practices in which security considerations often remain merely an afterthought. Consequently, many devices are released with inherent security vulnerabilities that can be exploited, which has led to an entirely new category of malware explicitly targeting IoT devices [1]–[4].

The preferred way to cope with vulnerabilities are security patches for the affected devices [5]. However, many devices lack appropriate facilities for automated updates or there may be significant delays until device manufacturers provide them, mandating the use of reactive security measures like intrusion detection systems (IDS) for detecting possible device compromise [6]–[9]. Signature-based IDSs look for specific communication patterns, so-called *attack signatures*, associated with known attacks. Such systems are, however, unable to detect novel attacks until the IDS vendor releases attack signatures for them [6].

To detect previously unknown attacks *anomaly detection* needs to be used which works by profiling the normal behavior of devices and detecting attacks as deviations from this normal behavior profile [7]–[9]. However, this approach often suffers from a high false alarm rate, making it unusable in practice. This problem is exacerbated in the IoT setting: First, there are hundreds of very heterogeneous devices on the market, which making it challenging to train a precise detection model covering all behaviors of various IoT devices. Second, IoT devices do typically not (notwithstanding a few exceptions) generate a lot of network traffic, as their communications are limited to, e.g., status updates about sensor readings or (relatively) infrequent interactions related to user commands. This scarcity of communications makes it challenging to train comprehensive models that can accurately cover the full behavior of IoT devices.

An effective anomaly detection model needs to capture *all* benign patterns of behavior to be able to differentiate them from malicious actions. The ever-increasing number of literally thousands of types of IoT devices (ranging from temperature sensors and smart light bulbs to big appliances like washing machines) and the typical scarcity of their communications, makes an all-encompassing behavior model 1) tedious to learn and update, and 2) too broad to be effective at detecting subtle anomalies without generating many false alarms.

Goals and Contributions. To tackle the above challenges we present DÏoT, a system for detecting compromised IoT devices. It uses a novel *device-type-specific* anomaly detection approach to achieve accurate detection of attacks while generating almost no false alarms. Major IoT device vendors, including Cisco, assisted us formulating real-world settings for our solution and usage scenarios.

We make the following contributions:

- DÏoT, a self-learning distributed system for security monitoring of IoT devices (Sect. II) based on *device-type-specific detection models* for detecting anomalous device behavior:
  - It utilizes a novel anomaly detection approach based on representing network packets as symbols in a language allowing to use a language analysis technique to effectively detect anomalies (Sect. III).

2575-8411/19/$31.00 ©2019 IEEE
DOI 10.1109/ICDCS.2019.00080

Authorized licensed use limited to: Universidad Nacional de Ingeniería. Downloaded on October 20,2026 at 06:07:28 UTC from IEEE Xplore. Restrictions apply.

  - It is the first system to apply a *federated* learning approach for aggregating anomaly-detection profiles for intrusion detection (Sect. IV).
- Systematic and extensive experimental analysis using more than 30 off-the-shelf IoT devices, showing that DÏoT is fast (detection in $\approx 257\,\mathrm{ms}$) and effective ($95.6\,\%$ true positive rate, *zero false alarms*, i.e., $0\,\%$ false positive rate) (Sect. VI).
- An *Attack* dataset of network traffic generated by real off-the-shelf consumer IoT devices infected with real IoT malware (Mirai [1]) using which we evaluate the effectiveness of DÏoT (Sect. V-A).

We will make our datasets as well as the DÏoT implementation available for research use.

II. System Model

Our system model is shown in Fig. 1. We consider a typical SOHO (Small Office/Home Office) network, where IoT devices connect to the Internet via an access gateway.

A. System Architecture

The DÏoT system consists of *Security Gateway* and *IoT Security Service*. The role of *Security Gateway* is to monitor devices and perform anomaly detection in order to identify compromised devices in the network. It is supported by *IoT Security Service*, which can be, e.g., a service provider like Microsoft, Amazon or Google that aggregates device-type-specific anomaly detection models trained by all *Security Gateway*s in the system.

1) Security Gateway: acts as the local access gateway to the Internet to which IoT devices connect over WiFi or Ethernet. It hosts the *Anomaly Detection* component. When a new device is added to the network, *Security Gateway* identifies its type as outlined in Sect. II-C. The *Anomaly Detection* component monitors the communications of identified IoT devices and detects abnormal communication behavior that is potentially caused by malware (Sect. III) based on anomaly detection models it trains locally and which are aggregated by the *IoT Security Service* to a global detection model.

2) IoT Security Service: supports *Security Gateway* by maintaining a repository of device-type-specific *anomaly detection models*. When a new device is added to the local network, *Security Gateway* identifies its *device type* and retrieves the corresponding anomaly detection model for this type from *IoT Security Service*. *IoT Security Service* also aggregates updates to device-type-specific anomaly detection models provided by the *Security Gateway*s in the system.

B. Adversary Model and Assumptions

Adversary. The adversary is IoT malware performing attacks against, or launching attacks from, vulnerable devices in the SOHO network. Hereby we consider all actions undertaken by the malware that it performs to discover, infect and exploit vulnerable devices as discussed in detail in Sect. V-A3.

Defense goals. The primary goal of DÏoT is to detect attacks on IoT devices in order to take appropriate countermeasures, e.g., by preventing targeted devices from being compromised or isolating compromised devices from the rest of the network. We aim to detect attacks at the earliest stage possible, preferably even *before* a device can be successfully infected.

In addition, we make following assumptions:

- **A1 - No malicious manufacturers.** IoT devices may be vulnerable but are not compromised when first released by a manufacturer. Adversaries must first find a vulnerability and a way to exploit it, which takes some time during which non-compromised devices generate only legitimate communications, leaving sufficient time (cf. Sect. VI-C) to learn benign models of device behavior.
- **A2 - Security Gateway is not compromised.** Since *Security Gateway* is the device enforcing security in the SOHO network, we assume that it is not compromised. Like firewall devices or antivirus software, if *Security Gateway* is compromised the SOHO network stops being protected by it. Several approaches can be used to protect it. For instance, if *Security Gateway* supports a suitable trusted execution environment, like Intel SGX [10] or trusted platform module, its integrity can be remotely verified using *remote attestation* techniques [11].
- **A3 - Automated identification of IoT devices.** A technique for automatically identifying and labeling IoT devices in the local SOHO network must be available. This technique should be implementable in the *Security Gateway* to identify IoT devices connected to it.

C. Device-Type Identification

As DÏoT uses device-type specific anomaly detection models, it requires the possibility to identify the *type of devices* in the network. Several solutions have been designed to automatically identify and label unknown IoT devices in a network [12]–[14]. Alternatively, manufacturer-provided explicit device-type specifications like MUD [15] or manual labeling of IoT devices could be used. For DÏoT, we selected an existing approach - AuDI [14] that autonomously identifies the *type* of individual IoT devices in a local network. This approach is accurate and fast (requiring only 30 minutes to identify device type at the accuracy of $98.2\,\%$). This approach considers *abstract device types* representing families of similar devices from the same device manufacturer with similar hardware and software configurations, resulting in highly identical communication behavior. It can be trained *without the need to manually label* communication traces of pre-defined real-world device types since it works by clustering device fingerprints so that each cluster can be automatically labeled with an abstract label, e.g., $type\#k$ which represents a specific IoT device type. It justifies our assumption A3 as mentioned above.

Using this approach we can reliably map devices to a corresponding *device type* for which DÏoT can build a device-type-specific model of normal behavior that can be used to effectively detect anomalous deviations. This allows DÏoT to be trained and operated autonomously, *without the need for human intervention at any stage*.

Authorized licensed use limited to: Universidad Nacional de Ingeniería. Downloaded on October 20,2026 at 06:07:28 UTC from IEEE Xplore. Restrictions apply.

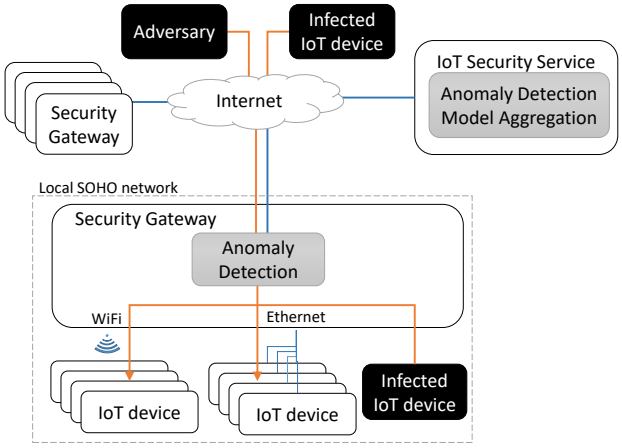


Fig. 1: DÏoT system model

D. Challenges

Anomaly detection techniques face challenges in the IoT application scenario:

- **C1- Dynamic threat landscape.** New IoT devices are released on a daily basis. A significant fraction of them have security vulnerabilities. Exploits targeting vulnerable devices are also being developed by adversaries at a similarly high pace. This makes the threats against IoT devices highly dynamic and ever-increasing.
- **C2- Resource limitations.** IoT devices have limited capabilities w.r.t. available memory, computing resources and energy often making it infeasible to perform on-device detection.
- **C3- IoT device heterogeneity and false alarms.** Behaviors of different IoT devices are very heterogeneous, so that anomaly detection techniques easily raise false alarms. However, to be useful in practice, anomaly detection systems must minimize false alarms.
- **C4- Scarcity of communications.** In contrast to high-end devices, IoT devices generate only little traffic, often triggered by infrequent user interactions.

E. Design Choices

1) Gateway monitoring: As on-device monitoring is rarely feasible due to challenge C2, we perform monitoring of IoT device communications on *Security Gateway*.

2) Device-type-specific anomaly detection: Since different IoT devices can have very heterogeneous behaviors (challenge C3), we model each device type's behavior with a dedicated model. Consequently, each model needs to cover only the behavior of a specific device type. As IoT devices are typically single-use appliances with only a few different functions, their behavior patterns are relatively static and limited, allowing the model to accurately capture all possible legitimate behaviors of a device type. Thus, the model is less prone to trigger false alarms (details in Sect. VI-D), thereby effectively addressing challenge C3.

3) Federated learning approach: Anomaly detection models are learned using a *federated learning approach* where *Security Gateway*s use locally collected data to train local models which *IoT Security Service* aggregates into a global model (details in Sect. IV). This aggregation maximizes the usage of limited information obtained from scarce communications at each gateway (challenge C4) and helps to improve the accuracy of anomaly detection models by utilizing all available data for learning.

4) Autonomous self-learning: Anomaly detection models are trained using data autonomously labeled with the device-type that generated it. Device types are also learned and assigned in an autonomous manner. The whole process does therefore not require any human intervention, which allows DÏoT to respond quickly and autonomously to new threats, addressing challenge C1. It is worth noting that DÏoT starts operating with no anomaly detection model. It learns and improves these models as *Security Gateway*s aggregate more data.

5) Modeling techniques requiring little data: As discussed in detail in Sect. III, we select features and machine learning algorithms (GRU) that can be efficiently trained even with few training data. This design choice addresses challenge C4.

III. Device-Type-Specific Anomaly Detection

Our anomaly detection approach is based on evaluating the communication patterns of a device to determine whether it is consistent with the learned benign communication patterns of that particular device type. The detection process is shown in Fig. 2. In **Step 1** the communication between the *Security Gateway* and the IoT device is captured as a sequence of packets $pkt_1, pkt_2, \ldots$. Each packet pkt_i is then in **Step 2** mapped to a corresponding symbol s_i characterizing the type of the packet using a mapping that is based on distinct characteristics $c_1, c_2, \ldots, c_7$ derived from each packet's header information as discussed in Sect. III-A. The mapped sequence of symbols $s_1, s_2, \ldots$ is then in **Step 3** input into a pre-trained model using Gated Recurrent Units (GRUs) [16], [17]. The GRU model will calculate a probability estimate p_i for each symbol s_i based on the sequence of k preceding symbols $s_{i-k}, s_{i-k+1}, \ldots, s_{i-1}$. GRU is a novel approach to recurrent neural networks (RNN) currently being a target of lively research. GRUs provide similar accuracy as other RNN approaches but are computationally less expensive [17], [18]. In **Step 4** the sequence of occurrence probability estimates $p_1, p_2, \ldots$ is evaluated to determine possible anomalies. If the occurrence probabilities p_i of a sufficient number of packets in a window of consecutive packets fall below a detection threshold, as described in detail in Sect. III-B, the packet sequence is deemed anomalous and an alarm is raised.

A. Modelling Packet Sequences

Data packets pkt_i in the packet sequence $pkt_1, pkt_2, \ldots$ emitted by an IoT device are mapped into *packet symbols* s_i based on 7-tuples $(c_1, c_2, \ldots, c_7)$ of discrete packet characteristics c_i of packet pkt_i. This mapping is defined by

Authorized licensed use limited to: Universidad Nacional de Ingeniería. Downloaded on October 20,2026 at 06:07:28 UTC from IEEE Xplore. Restrictions apply.

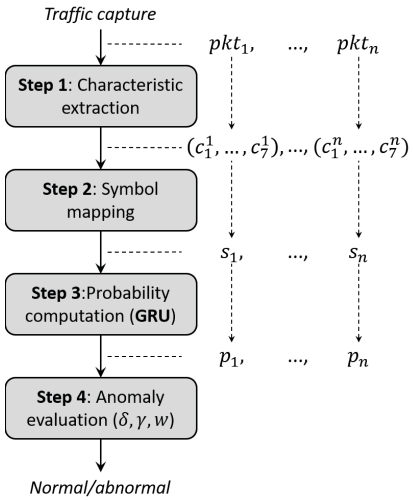
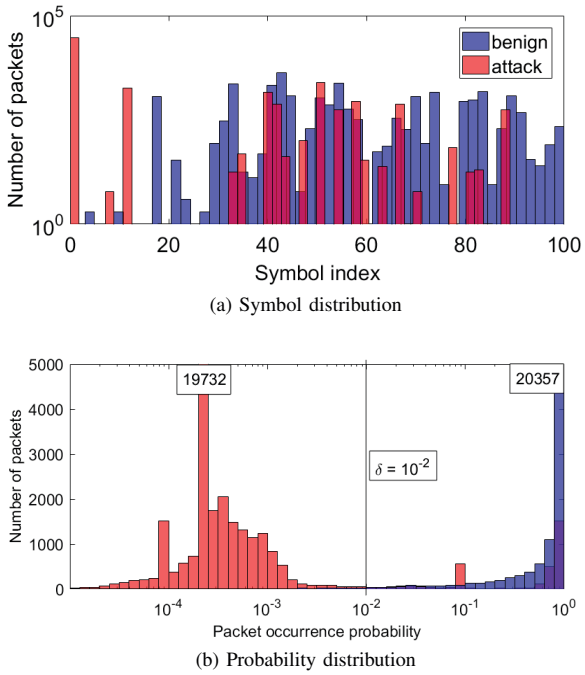


Fig. 2: Overview of device-type-specific anomaly detection

a device-type-specific mapping function $mapping_{type\#k}$: $A \rightarrow B_{type\#k}$ s.t. $mapping_{type\#k}(pkt_i) = s_i$ where A is the domain of raw network packets pkt and $B_{type\#k}$ is the domain of packet symbols s for device-type $type\#k$. Mapping $mapping_{type\#k}$ assigns each unique combination of packet characteristics $(c_1, \ldots, c_7)$ a dedicated symbol s representing the 'type' of the particular packet.

We use the following packet characteristics shown also in Tab. I:

- c_1 **direction:** (*incoming / outgoing*) Normal TCP traffic is usually balanced two-way communication but abnormal is not as, e.g., a bot only sends packets to a victim without receiving replies when running DDoS attacks.
- c_2 **and** c_3 **local and remote port type:** (*system / user / dynamic*) Each device-type uses specific ports designed by the manufacturers while malicious attack patterns usually use different ports.
- c_4 **packet length:** (*bin index of packet's length where eight most frequently occurring packet lengths receive dedicated bins and one bin for other packet length values*) Each device-type communicates using specific packet patterns with specific packet lengths that are mostly different in malicious attack patterns.
- c_5 **TCP flags:** Normal communications contain packets with specific TCP flag sequences e.g., $SYN \rightarrow SYNACK \rightarrow ACK \rightarrow PUSH \rightarrow FIN$. However, many attacks do not follow standard protocols, e.g., SYN flood (DDoS attack) only sends SYN messages.
- c_6 **encapsulated protocol types:** Each device type usually uses a set of specific protocols, which is likely different from protocol types used in attacks.
- c_7 **IAT bin:** (*bin index of packet inter-arrival time (IAT) using three bins:* $< 0.001\,\text{ms}$, $0.001\,\text{ms}$ *to* $0.05\,\text{ms}$, *and* $> 0.05\,\text{ms}$) Many attacks (e.g., DDoS) usually generate traffic at a high packet rate, resulting in smaller IAT values in than normal communications.



(a) Symbol distribution

(b) Probability distribution

Fig. 3: Packet symbol occurrence frequencies and occurrence probability estimates for benign and attack traffic for Edimax smart power plugs

TABLE I: Packet characteristics used in symbol mapping

ID	Characteristic	Value
c_1	direction	$1 =$ incoming, $0 =$ outgoing
c_2	local port type	bin index of port type
c_3	remote port type	bin index of port type
c_4	packet length	bin index of packet length
c_5	TCP flags	TCP flag values
c_6	protocols	encapsulated protocol types
c_7	IAT bin	bin index of packet inter-arrival time

B. Detection Process

Fig. 3a shows an example of the occurrence frequencies of individual packet symbols for benign and attack traffic (as generated by the Mirai malware) for Edimax smart power plugs. It can be seen that using packet symbols alone to distinguish between benign and attack traffic is not sufficient, as both traffic types contain packet types that are mapped to the same symbols. Our detection approach is therefore based on estimating the likelihood of observing individual packet types given the sequence of preceding packets. The rationale behind this approach is the observation that IoT device communications usually follow particular characteristic patterns. Traffic generated by IoT malware, however, doesn't follow these patterns and can therefore be detected. We will thus use the detection model to calculate an *occurrence probability* p_i

Authorized licensed use limited to: Universidad Nacional de Ingeniería. Downloaded on October 20,2026 at 06:07:28 UTC from IEEE Xplore. Restrictions apply.

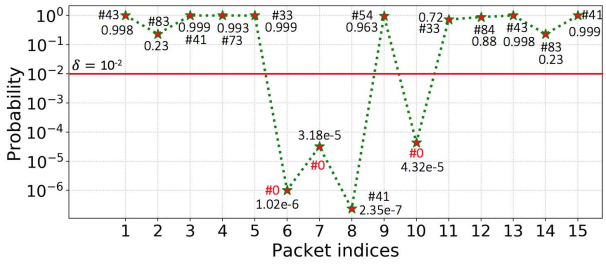


Fig. 4: Occurrence probabilities of 15 packets from Edimax Plug when Mirai was in standby stage. The red '#0' denotes the malicious packets.

for each packet symbol s_i given the sequence of k preceding symbols $< s_{i-k}, s_{i-k+1}, \ldots, s_{i-1} >$, i.e.,

$$p_i = P(s_i | < s_{i-k}, s_{i-k+1}, \ldots, s_{i-1} >) \quad (1)$$

Parameter k is a property of the used GRU network and denotes the length of the lookback history, i.e., the number of preceding symbols that the GRU takes into account when calculating the probability estimate. From Fig. 3b we can see that these probability estimates are on average higher for packets belonging to benign traffic patterns, and lower for packets generated by malware on an infected device and can therefore be flagged as anomalous.

Definition 1 (Anomalous packets): Packet pkt_i mapped to packet symbol s_i is *anomalous*, if its occurrence probability p_i is below *detection threshold* δ, i.e., if

$$p_i < \delta \quad (2)$$

We performed an extensive empirical analysis of the probability estimates provided by device-specific detection models for both benign and attack traffic for the datasets described in Sect. VI and could determine that a value of $\delta = 10^{-2}$ provides a good separation between benign and attack traffic, as can be also seen in Fig. 3b. An example of our approach is shown in Fig. 4. Malicious packets (represented by symbol '#0') get very low probability estimates ($<10^{-4}$), distinguishing them clearly from benign packets. However, their presence at indices $6-7$ also affects the estimate of the benign packet '#41' at index 8 ($<10^{-6}$), since the sequence of packets preceding this packet is unknown to the detection model.

Triggering an anomaly each time an anomalous packet is observed would lead to numerous false positive detections, as also benign traffic may contain noise that is not covered by the GRU model and will therefore receive low occurrence probability estimates. An anomaly is therefore triggered only in the case that a significant number of packets in a window of consecutive packets are anomalous.

Definition 2 (Anomaly triggering condition): Given a window W of w consecutive packets $W = (pkt_1, pkt_2, \ldots, pkt_w)$ represented by symbol sequence $S = (s_1, s_2, \ldots s_w)$, we trigger an anomaly alarm, if the fraction of anomalous packets in W is larger than an anomaly triggering threshold γ, i.e., if

$$\frac{|\{s_i \in S | p_i < \delta\}|}{w} > \gamma \quad (3)$$

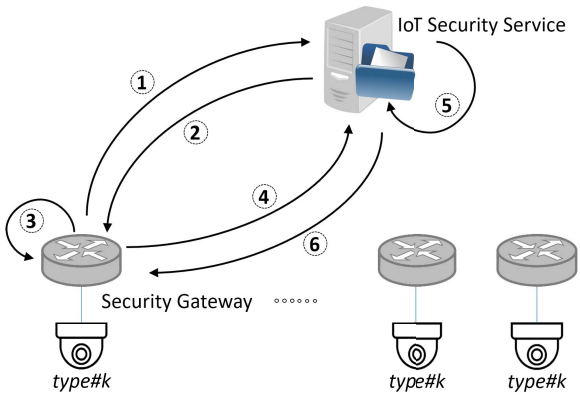


Fig. 5: Overview of federated learning process

IV. Federated Learning Approach

The GRU models are learned using traffic collected at several *Security Gateway*s, each monitoring a client IoT network. Each *Security Gateway* observing a device of a particular type $type\#k$ contributes to training its anomaly detection model. We take a *federated learning* approach to implement the distributed learning of models from several clients. Federated learning is a communication-efficient and privacy-preserving learning approach suited for distributed optimization of Deep Neural Networks (DNN) [19], [20]. In federated learning, clients do not share their training data but rather train a local model and send model updates to a centralized entity which aggregates them. Federated learning is chosen because it is suitable [21] for scenarios where:

- data are massively distributed, so that there is a large number of clients each having a small amount of data. IoT devices typically generate little traffic, which means only little data can be provided by each client alone.
- contributions from clients are imbalanced. In our system, the training data available at each *Security Gateway* depends on the duration that an IoT device has been in the network and the amount of interaction it has had, which varies largely between clients.

A. Learning Process

The federated training process is illustrated in Fig. 5. Each *Security Gateway* having devices of a particular type $type\#k$ in its network requests a detection profile for this type from *IoT Security Service* in **Step 1** and gets an initial GRU model for this type in **Step 2**. At the start of DÏoT, this model is random, otherwise it is already trained through several rounds of the following process. In **Step 3** the global model is retrained locally by each *Security Gateway* with traces collected by monitoring communication of the $type\#k$ devices. Then

Authorized licensed use limited to: Universidad Nacional de Ingeniería. Downloaded on October 20,2026 at 06:07:28 UTC from IEEE Xplore. Restrictions apply.

in **Step 4** local updates made to the model by each *Security Gateway* are reported to *IoT Security Service* which in **Step 5** aggregates them as defined in Def.3 [21] to improve the global model. Finally, the updated global model for $type\#k$ devices is then pushed back to *Security Gateway* and used for anomaly detection (**Step 6**). The re-training of the model is performed on a regular basis to improve its accuracy.

Definition 3 (Global Model Aggregation): Given n clients with their associated model weights $W_1, \ldots, W_n$ trained by associated number of data samples $s_1, \ldots, s_n$. We define the global model G which is aggregated from those local models as follows:

$$G = \sum_{i=1}^{n} \frac{s_i}{s} W_i \quad (\text{where } s = \sum_{i=1}^{n} s_i) \tag{4}$$

To train our models we adopt an approach introduced by McMahan *et al.* [21]. Each client (*Security Gateway*) trains its GRU model locally for several epochs before reporting updates to *IoT Security Service*. This limits the communication overhead by reducing the number of updates to send to the *IoT Security Service*. To the best of our knowledge we are the first to employ a federated learning approach for anomaly detection-based intrusion detection.

B. Federated Learning Setup

We implemented the federated learning algorithm utilizing the *flask* [22] and *flask_socketio* [23] libraries for the server-side application and the *socketIO-client* [24] library for the client-side application. The socketIO-client uses the *gevent* asynchronous framework [25] which provides a clean API for concurrency and network related tasks. We used the *Keras* [26] library with *Tensorflow* backend to implement the GRU network with the parameters selected in Sect. V-B.

V. Experimental Setup

To evaluate DÏoT, we apply it on the use case of detecting real-life IoT malware. We selected Mirai for this purpose, since its source code is publicly available and several infamous malware variants like Hajime [2] or Persirai [27] have been implemented using the same code base. Mirai also realizes similar attack stages (detailed in Sect. V-A3 below) as state-of-the-art IoT malware [3], [28]. This makes Mirai a highly relevant baseline for IoT malware behavior.

A. Datasets

We collected extensive datasets about the communication behavior of IoT devices in laboratory and real-world deployment settings. The monitored devices included 33 typical consumer IoT devices like IP cameras, smart power plugs and light bulbs, sensors, etc. The devices were mapped by our device-type-identification method to 23 unique device types. The detailed list of devices and assignment to device-types can be found in Tab. II. We collected datasets by setting up a laboratory network as shown in Fig. 6 using `hostapd` on a laptop running Kali Linux to create a gateway acting as an access point with WiFi and Ethernet interfaces to which

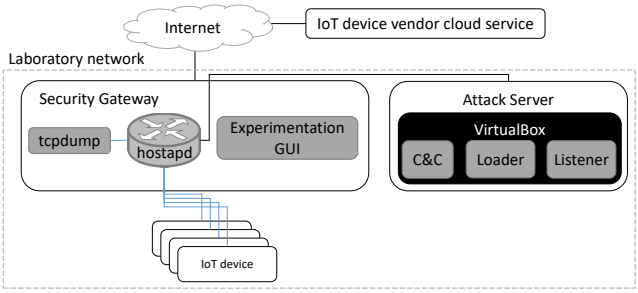


Fig. 6: Laboratory network setup

IoT devices were connected. On the gateway we collected all network traffic packets originating from the monitored devices using `tcpdump`.

1) Activity dataset: A key characteristic of IoT devices is that they expose only a few distinct actions accessible to users, e.g., ON, OFF, ADJUST, etc. To capture the communication patterns related to user interactions with IoT devices, we collected a dataset encompassing all such actions being invoked on the respective IoT devices. We repeatedly performed actions shown in Tab. IV. Each of the actions was repeated 20 times (20-time repetition chosen as a rule of thumb). To capture also less intensive usage patterns, the dataset was augmented with longer measurements of two to three hours, during which actions were triggered only occasionally. This dataset contains data from 33 IoT devices out of which 27 have both action and standby data. Six devices (lighting and home automation hubs) have standby data only because they do not provide meaningful actions that users could invoke.

2) Deployment dataset: To evaluate DÏoT in a realistic smart home deployment setting, in particular with regard to how many false alarms it will raise, we installed a number of ($n = 14$) different smart home IoT devices[1] in several different domestic deployment scenarios. This deployment involved real users and collected communication traces of these devices under realistic usage conditions. We used the same set-up as in the laboratory network for the domestic deployment, albeit we excluded the attack server. Users used and interacted with the IoT devices as part of their everyday life. Packet traces were collected continuously during one week.

3) Attack dataset: For evaluating the effectiveness of DÏoT at detecting attacks, we collected a dataset comprising malicious traffic of IoT devices infected with Mirai malware [1], [3] in all four different attack stages discussed below: *pre-infection*, *infection*, *scanning* and *DoS attacks* (as a monetization stage). Additionally, we collected traffic when Mirai was in a *standby* mode, i.e., not performing any attack but awaiting commands from its *Command & Control* server.

Among 33 experimental devices, we found 5 devices which are vulnerable to the Mirai malware. The *Attack* dataset was collected from those five devices: *D-LinkCamDCS930L*,

[1]The number of devices was limited, as the driver of the used WiFi interface allowed at most 16 devices to reliably connect to it simultaneously.

Authorized licensed use limited to: Universidad Nacional de Ingeniería. Downloaded on October 20,2026 at 06:07:28 UTC from IEEE Xplore. Restrictions apply.

TABLE II: 33 IoT devices used in the *Activity*, *Deployment* and *Attack* datasets and their connectivity technologies + Affectation of these devices to 23 DÏoT device types during evaluation.

Device-type	Identifier	Device model	WiFi	Ethernet	Other	Activity	Deployment	Attack
type#01	ApexisCam	Apexis IP Camera APM-J011	•	•	○	•	○	○
type#02	CamHi	Cooau Megapixel IP Camera	•	•	○	•	○	○
type#03	D-LinkCamDCH935L	D-Link HD IP Camera DCH-935L	•	○	○	•	○	○
type#04	D-LinkCamDCS930L	D-Link WiFi Day Camera DCS-930L	•	•	○	•	•	•
	D-LinkCamDCS932L	D-Link WiFi Camera DCS-932L	•	•	○	•	•	•
type#05	D-LinkDoorSensor	D-Link Door & Window sensor	○	○	•	•	○	○
	D-LinkSensor	D-Link WiFi Motion sensor DCH-S150	•	○	○	•	•	○
	D-LinkSiren	D-Link Siren DCH-S220	•	○	○	•	○	○
	D-LinkSwitch	D-Link Smart plug DSP-W215	•	○	○	•	•	○
	D-LinkWaterSensor	D-Link Water sensor DCH-S160	•	○	○	•	○	○
type#06	EdimaxCamIC3115	Edimax IC-3115W Smart HD WiFi Network Camera	•	•	○	•	○	○
	EdimaxCamIC3115(2)	Edimax IC-3115W Smart HD WiFi Network Camera	•	•	○	•	○	○
type#07	EdimaxPlug1101W	Edimax SP-1101W Smart Plug Switch	•	•	○	•	•	•
	EdimaxPlug2101W	Edimax SP-2101W Smart Plug Switch	•	○	○	•	•	•
type#08	EdnetCam	Ednet Wireless indoor IP camera Cube	•	•	○	•	○	○
type#09	EdnetGateway	Ednet.living Starter kit power Gateway	•	○	•	•	•	○
type#10	HomeMaticPlug	Homematic pluggable switch HMIP-PS	○	○	•	•	○	○
type#11	Lightify	Osram Lightify Gateway	•	○	•	•	•	○
type#12	SmcRouter	SMC router SMCWBR14S-N4 EU	•	•	○	•	○	○
type#13	TP-LinkPlugHS100	TP-Link WiFi Smart plug HS100	•	○	○	•	○	○
	TP-LinkPlugHS110	TP-Link WiFi Smart plug HS110	•	○	○	•	○	○
type#14	UbnTAirRouter	Ubnt airRouter HP	•	•	○	•	○	•
type#15	WansviewCam	Wansview 720p HD Wireless IP Camera K2	•	○	○	•	○	○
type#16	WeMoLink	WeMo Link Lighting Bridge model F7C031vf	•	○	•	•	○	○
type#17	WeMoInsightSwitch	WeMo Insight Switch model F7C029de	•	○	○	•	○	○
	WeMoSwitch	WeMo Switch model F7C027de	•	○	○	•	○	○
type#18	HueSwitch	Philips Hue Light Switch PTM 215Z	○	○	•	•	•	○
type#19	AmazonEcho	Amazon Echo	•	○	○	•	•	○
type#20	AmazonEchoDot	Amazon Echo Dot	•	○	○	•	○	○
type#21	GoogleHome	Google Home	•	○	○	•	•	○
type#22	Netatmo	Netatmo weather station with wind gauge	•	○	•	•	•	○
type#23	iKettle2	Smarter iKettle 2.0 water kettle SMK20-EU	•	○	○	•	•	○
	SmarterCoffee	Smarter SmarterCoffee coffee machine SMC10-EU	•	○	○	•	•	○

TABLE III: Characteristics of used datasets

Dataset (Number of devices)	Time (hours)	Size (MiB)	Flows	Packets
Activity (33)	165	465	115,951	2,087,280
Deployment (14)	2,352	578	95,518	2,286,697
Attack (5)	84	7,734	8,464,434	21,919,273

TABLE IV: Actions for different IoT device categories

Category (count)	Typical actions
IP cameras (6)	START / STOP video, adjust settings, reboot
Smart plugs (9)	ON, OFF, meter reading
Sensors (3)	trigger sensing action
Smart lights (4)	turn ON, turn OFF, adjust brightness
Actuators (1)	turn ON, turn OFF
Appliances (2)	turn ON, turn OFF, adjust settings
Routers (2)	browse amazon.com
Hub devices (6)	no actions

D-LinkCamDCS932L, *EdimaxPlug1101W*, *EdimaxPlug2101W* and *UbntAirRouter*. This was done by installing the *Command & Control*, *Loader* and *Listener* server modules on the laboratory network for infecting target devices with Mirai and controlling them. Infection was achieved using security vulnerabilities like easy-to-guess default passwords to open a terminal session to the device and issuing appropriate commands to download the malware binary onto the device.

In the *pre-infection* stage, *Loader* sends a set of commands via telnet to the vulnerable IoT device to prepare its environment and identify an appropriate method for uploading the Mirai binary files. We repeated the pre-infection process 50 times for each device. During each run, around 900 pre-infection-related packets were generated.

After pre-infection the *infection stage* commences, during which *Loader* uploads Mirai binary files to the IoT device. It supports three upload methods: `wget`, `tftp` and `echo` (in this priority order). To infect the two D-Link cameras and the Ubnt router *Loader* uses `wget`, on the Edimax plugs it will resort to using `tftp` as these are installed on the devices by default. We repeated the infection process 50 times for each device, each run generating approximately 700 data packets.

In the *scanning stage* we collected packets while the infected devices were actively performing a network scan in order to locate other vulnerable devices. Data collection was performed for five minutes per device, resulting in a dataset of more than 446,000 scanning data packets.

We extensively tested the *DoS attack stage*, utilizing all ten different DoS attack vectors (for details, see [29]) available in the Mirai source code [30]. We ran all attacks separately on

Authorized licensed use limited to: Universidad Nacional de Ingeniería. Downloaded on October 20,2026 at 06:07:28 UTC from IEEE Xplore. Restrictions apply.

all five compromised devices for five minutes each, generating more than 20 million packets of attack traffic in total.

Tab. III summarizes the sizes and numbers of distinct packets and packet flows in the different datasets. While packet flows can't be directly mapped to distinct device actions, they do provide a rough estimate of the overall level of activity of the targeted devices in the dataset.

B. Parameter Selection

Based on initial experiments with our datasets (Tab. III) we inferred that a lookback history of $k = 20$ symbols is sufficient to capture most communication interactions with sufficient accuracy. We used a GRU network with three hidden layers of size 128 neurons each. The size of the input and output layers is device-type-specific and equal to the number of mapping symbols of the function $mapping_{type\#k}$, which is equal to $\mid B_{type\#k} \mid$ (cf. Sect. III-A). We learned 23 anomaly detection models, each corresponding to a device type identified using the method described in Sect. II-C. Each anomaly detection model was trained with, and respectively tested on, communication from all devices matching the considered type.

C. Evaluation Metrics

We use false positive and true positive rate (FPR and TPR) as measures of fitness. FPR measures the rate at which benign communication is incorrectly classified as anomalous by our method causing a false alarm to be raised. TPR is the rate at which attacks are correctly reported as anomalous. We seek to minimize FPR, since otherwise the system easily becomes unusable, as the user would be overwhelmed with false alarms. At the same time we want to maximize TPR so that as many attacks as possible will be detected by our approach.

Testing for false positives was performed by four-fold cross-validation for device types in the *Activity* and *Deployment* datasets. The data were divided equally into four folds using three folds for training and one for testing. To determine the FPR, we divided the testing dataset according to Def. 2 into windows of $w = 250$ packets. Since the testing data contained only benign communications, any triggered anomaly alarm for packets of the window indicated it as a false positive, whereas windows without alarms were considered a true negative.

Testing for true positives was done by using the *Activity* and *Deployment* datasets as training data and the *Attack* dataset for testing with the same settings as for false positive testing. Moreover, as we know that the *Attack* dataset also contains benign traffic corresponding to normal operations of the IoT devices, we were interested in the average duration until detection. Therefore, in each window of $w = 250$ packets we calculated the number of packets required until an anomaly alarm was triggered in order to estimate the average detection time. In terms of TPR, such windows were considered true positives, whereas windows without triggered alarms were considered false negatives.

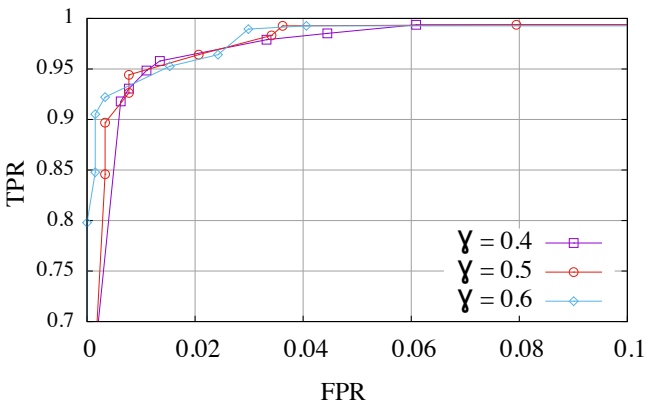


Fig. 7: ROC curve of TPR and FPR in dependence of detection threshold δ and anomaly triggering threshold γ.

VI. Experimental Results

A. Accuracy

To determine appropriate values for the detection threshold δ and anomaly triggering threshold γ, we evaluated FPR using the *Activity (33 devices, 23 device types)* dataset and TPR using the *Attack (5 devices, 3 device types)* dataset for a fixed window size of $w = 250$. Fig. 7 shows the receiver operating characteristic (ROC) curve of FPR and TPR in dependence of these parameters. We can see that all curves quickly reach over 0.9 TPR while keeping a very low FPR (<0.01), which is one of the main objectives for our approach. We therefore select $\delta = 0.01$ and $\gamma = 0.5$ at $w = 250$, which achieves 94.01% TPR at <0.01 FPR.

Using these selected parameters in the *Deployment (14 devices, 10 device types)* dataset and *Attack (5 devices)* dataset, we achieved an attack detection rate of 95.60% TPR and **no false positives**, i.e., 0% FPR during one week of evaluation. These results show that DÏoT can successfully address challenge C3, *reporting no false alarms in a real-world deployment setting*. Tab. V shows the detailed performance of our system for different attack scenarios (cf. Sect. V). The time to detect attacks varies according to the traffic intensity of the attacks. The average detection delay over all tested attacks is 257 ± 194 ms. DÏoT can detect an attack in the pre-infection stage after 223 packets while Mirai generates more than 900 packets during pre-infection. It means DÏoT is able to detect the attack *even before the attack proceeds to the infection stage*.

The detection rate for DoS attacks is lower than for other attack stages. However, *all DoS attacks are eventually detected* because DoS attacks have a high throughput ($1,412.94$ packets/s.) and we analyze five windows of 250 packets per second at this rate. Considering the 88.96% TPR we achieve on DoS attacks, four windows out of five are detected as anomalous and trigger an alarm. It is also worth noting that infected devices in *standby* mode get detected in *33.33%* of cases, while this activity is very stealthy (0.05 packets/s).

Authorized licensed use limited to: Universidad Nacional de Ingeniería. Downloaded on October 20,2026 at 06:07:28 UTC from IEEE Xplore. Restrictions apply.

TABLE V: Average detection times of analyzed Mirai attacks

Attack	packets/s.	det. time (ms.)	TPR
standby	*0.05*	*4,051,889*	*33.33%*
Pre-Infection	426.66	524	100.00%
Infection	721.18	272	93.45%
Scanning	752.60	166	100.00%
DoS	1,412.94	92	88.96%
Average	866.88	257±194	95.60%

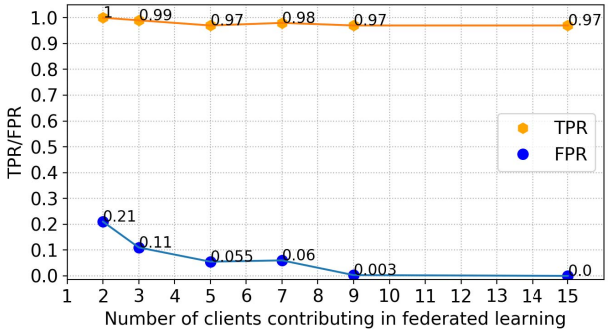


Fig. 8: Evolution of TPR and FPR as we increase the number of clients in federated learning. TPR decreases slightly (-3%) while FPR reaches 0 (-21%) when using 15 clients.

B. Efficiency of Federated Learning

We conducted a set of experiments to evaluate federated learning performance with different numbers of clients (ranging from 2 to 15) contributing to the training of the models. We selected the *number of epochs* that each client trains its local model to be 17 and specified the *number of communication rounds* between clients and server to be 3. Therefore, the local models were trained a total of 51 epochs. This was deemed sufficient since in our initial experiments utilizing a centralized learning setting the models converged after approximately 50 epochs. Each client was allocated a randomized subset of training data from the *Deployment* dataset (ranging from 0.1% to 10% of the total training dataset size) and we evaluated the system's performance for different numbers of clients involved in building the federated model. In average, each client takes approximately one second to train one local epoch on its data. We repeated our experiment three times for each device type, with random re-sampling of the training datasets. As expected, Fig. 8 shows that the federated models with more participating clients achieve better FPR, while TPR deteriorates only slightly.

Federated learning provides better privacy for clients contributing to training as they do not need to share their training data. However, this may result in a loss of accuracy of the obtained model in comparison to training the model in a centralized manner. To evaluate this possible loss in accuracy, we trained three federated models using the entire training dataset by dividing it among 5, 9 or 15 clients and comparing these to a model trained in a centralized manner. Tab. VI shows a small decrease in TPR as we increase the number of clients

TABLE VI: Effect of using federated learning comparing to centralizing approach

Type	Centralized learning	Federated learning		
		5 clients	9 clients	15 clients
FPR	0.00%	0.00%	0.00%	0.00%
TPR	95.60%	95.43%	95.01%	94.07%

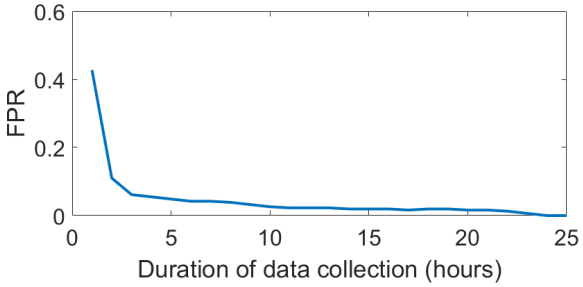


Fig. 9: Effect of training data size in time to FPR

while FPR is not deteriorated (remaining constant at 0.00%). This small drop in TPR is not a concern since a large number of packet windows would still trigger an alarm for any attack stage.

C. Data Needed for Training

Fig. 9 shows an example of detection model performance for two Edimax smart plug devices (models 1101W & 2101W) in dependency of the amount of data used for training the model. We divided the 7-day *Deployment* dataset into one-hour data chunks and randomly sampled different amounts of data chunks for training the model, gradually increasing the training dataset size. The figure shows that the FPR decreases noticeably when the training dataset grows. More importantly, the model needs less than 25 hours of data to achieve FPR = 0. It shows that our detection model needs little data for training and it means DÏoT can address challenge C4. Moreover, with the help of our federated learning approach leveraging several clients contributing to training the model, each client needs only a small amount of data i.e, 2.5 hours if there are ten clients involved. It justifies our assumption A1 as mentioned in Sect. II-B.

D. Efficiency of Device-Type-Specific Models and Scalability

Traditional anomaly detection approaches utilizing a single model for modeling benign behavior easily suffer from increasing false positive rates or decreasing sensitivity when the number of different types of behaviors (i.e., device types) captured by the model grows. This makes them unsuitable for real-world deployments with hundreds or thousands of different device types. Our solution, however, does not have this drawback, as it uses a dedicated detection model for each device type (details in Sect. III). Each of these models focuses solely on the characteristic behavior of one single device type, resulting in more specific and accurate behavioral models, independent of the number of different device types handled

Authorized licensed use limited to: Universidad Nacional de Ingeniería. Downloaded on October 20,2026 at 06:07:28 UTC from IEEE Xplore. Restrictions apply.

by the system. To evaluate the benefit of using device-type-specific anomaly detection models compared to using a single model for all devices, we evaluated a single model on the whole *Deployment* dataset using 4-fold-cross validation and evaluated detection accuracy on the *Attack* dataset. The result is as expected: FPR increases from 0% to 0.67% while TPR increases from 95.6% to 97.21%. However, as mentioned in Sect. II, a high false alarm rate would make the anomaly detection system impractical. If the system had FPR of 0.67% in our deployment setup, it would trigger around eight alarms per day. It means a smarthome with dozens of devices could have hundreds of false alarms per day.

E. Performance

We evaluated the processing performance of GRU without specific performance optimizations on a laptop and a desktop computer. The laptop ran Ubuntu Linux 16.04 with an Intel©Core™i7-4600 CPU with 8GB of memory, whereas the desktop ran Ubuntu Linux 18.04 with an Intel©Core™i7-7700 CPU with 8GB of memory and a Radeon RX 460 core graphic card. We evaluated the processing performance of GRU without specific optimizations on a laptop and a desktop computer. The laptop ran Ubuntu Linux 16.04 with an Intel i7-4600 CPU with 8GB of memory, whereas the desktop ran Ubuntu Linux 18.04 with an Intel i7-7700 CPU with 8GB of memory and a Radeon RX 460 core graphics card with GPU. Average processing time per symbol (packet) for prediction was $0.081(\pm 0.001)$ ms for the desktop utilizing its GPU and $0.592(\pm 0.001)$ ms when executed on the laptop with CPU. On average, training a GRU model for one device type took 26 minutes on the desktop and 71 minutes on the laptop hardware when considering a week's worth of data in the *Deployment* dataset. We conclude from this that model training will be feasible to realize in real deployment scenarios, as training will in any case be done gradually as data are collected from the network over longer periods of time.

VII. Effectiveness

A. Generalizability of Anomaly Detection

Although we focused our evaluation on the most well-known IoT malware so far: Mirai [1] for the use case, DÏoT is likely effective also in detecting other botnet malware like Persirai [27], Hajime [2], etc. DÏoT's anomaly detection leverages deviations in the behavior of infected IoT devices caused by the malware. Such deviations will be observable for any malware.

B. Evolution of IoT Device Behavior

The behavior of an IoT device type can evolve due to, e.g., firmware updates that bring new functionality. This modifies its behavior and may trigger false alarms for legitimate communication. We prevent these false alarms by correlating anomaly reports from all *Security Gateway*s at the *IoT Security Service*. Assuming firmware updates would be propagated to many client networks in a short time, if alarms are reported from a large number of security gateways for the same device type in a short time, we can cancel the alarm and trigger re-learning of the corresponding device identification and anomaly detection models to adapt to this new behavior. To ensure that sudden widespread outbreaks of an IoT malware infection campaign are not erroneously interpreted as firmware updates, the canceling of an alarm can be confirmed by a human expert at the *IoT Security Service*. This should represent a small burden, as the roll-out of firmware updates is a relatively seldom event.

C. Mimicking Legitimate Communication

An adversary that has compromised an IoT device can attempt to mimic the device's legitimate communication patterns to try to remain undetected. However, as the device-type-specific detection model is restricted to the (relatively limited) functionality of the IoT device, it is in practice very difficult for the adversary to mimic legitimate communication and at the same time achieve a malicious purpose, e.g., scanning, flooding, etc., especially when considering that any change in packet flow semantics is also likely to change the characteristics (protocol, packet size, port, etc.) of packets and their ordering, which are both used for detecting anomalies in the packet sequence. Moreover, adversaries would need to know the device-type-specific communication patterns in order to mimic them. This makes it significantly harder for adversaries to develop large scale IoT malware that affects a wide range of different IoT device types in the way that, e.g., Mirai does.

D. Adversarial Machine Learning

Adversarial examples. If an adversary manages to compromise an IoT device while remaining undetected, it can attempt to 'poison' the training process of the system by forging packets as *adversarial examples* that are specifically meant to influence the learning of the model in such a way that malicious activities are not detected by it. There exist techniques to forge adversarial examples to neural networks [31]. However, these apply to images [32], [33] and audio inputs [34], [35], where objective functions for the underlying optimization problem are easy to define. Forging adversarial examples consists of finding minimal modifications ϵ for an input x of class C such that $x+\epsilon$ is classified as $C' \neq C$. For example, in our case this would mean that a malicious packet is incorrectly classified as a benign one. In contrast to image or audio processing, however, our features (symbols) are not raw but processed from packet properties. First, it means that modifications ϵ are computed for our symbolic representation of packet sequences which are difficult to realize in a way that would preserve their utility for the adversary, i.e., realize 'useful' adversarial functionality required for malicious activities like scanning or DoS. Second, it is difficult to define the objective distance to minimize in order to achieve "small modifications" since modifying the value of one single packet characteristic (protocol, port, etc.) can change the semantics of a packet entirely.

Authorized licensed use limited to: Universidad Nacional de Ingeniería. Downloaded on October 20,2026 at 06:07:28 UTC from IEEE Xplore. Restrictions apply.

Poisoning federated learning. For initial model training, we can assume the training data contain only legitimate network traffic, as devices are assumed initially to be benign (assumption A1 (Sect. II-B)). However, the federated setting can be subject to *poisoning attacks* during re-training, where the adversary uses adversarial examples as described above to corrupt the anomaly detection model so that it eventually will accept malicious traffic as benign [36] (or vice versa). Techniques have been developed for preventing poisoning attacks by using local outlier detection-based filtering of adversarial examples to pre-empt model poisoning [37].

In the scope of this paper we assume that the *Security Gateway* is not compromised by the adversary (assumption A2 (Sect. II-B)). However, since a malicious user can have physical access to his *Security Gateway*, it is thinkable that he could compromise it in order to stage a poisoning attack against the system using adversarial examples. In this case, local filtering of adversarial examples is not possible, as it can not be enforced by the compromised *Security Gateway*. We are therefore currently focusing our ongoing research efforts on applying poisoning mitigation approaches applied at the *IoT Security Service*. These include using more robust learning approaches less resilient to adversarial examples that will 'average out' the effects of adversarial examples, as well as approaches similar to, e.g., Shen *et al.* [38], where malicious model updates are detected before they are incorporated in global detection models.

VIII. Related Work

Several solutions have been proposed for the detection and prevention of intrusions in IoT networks [39], [40], sensor networks [9] and industrial control systems [41], [42]. SVELTE [40] is an intrusion detection system for protecting IoT networks from already known attacks. It adapts existing intrusion detection techniques to IoT-specific protocols, e.g., 6LoWPAN. Similarly, Doshi at al. [6] proposed a signature-based approach to detect known DDoS attacks using features representing the density of the network traffic. In contrast, DÏoT performs dynamic detection of any unknown attacks that deviate from the legitimate behavior of the device, since it only models legitimate network traffic. Jia *et al.* [39] proposed a context-based system to automatically detect sensitive actions in IoT platforms. This system is designed for patching vulnerabilities in appified IoT platforms such as Samsung SmartThings. It is not applicable to multi-vendor IoT systems while DÏoT is platform independent.

Detecting anomalies in network traffic has a long history [7], [8], [43], [44]. Existing approaches rely on analysing single network packets [7] or clustering large numbers of packets [8], [9] to detected intrusions or compromised services. Some works have proposed, as we do, to model communication as a language [42], [43]. For instance, authors of [43] derive finite state automatons from layer 3-4 communication protocol specifications. Monitored packets are processed by the automaton to detect deviations from protocol specification or abnormally high usage of specific transitions. Automatons can only model short sequences of packets while we use GRU to model longer sequences, which enables the detection of stealthy attacks. Also, modelling protocol specifications is coarse and leaves room for circumventing detection. In contrast, we use finer grained-features that are difficult to forge while preserving the adversarial utility of malicious packets.

Lately, recurrent neural networks (RNN) have been used for several anomaly-detection purposes. Most applications leverage Long Short-Term Memory (LSTM) networks for detecting anomalies in time series [45], aircraft data [46] or system logs [47]. Oprea at al. [48] use deep belief networks for mining DNS log data and detect infections in enterprise networks. In contrast to these works, DÏoT uses a different flavor of RNN, namely GRU that can be learned using less training data, enabling DÏoT to be trained faster, and operate in real-time, detecting anomalies in live network traffic instead of utilizing off-line analysis.

IX. Summary

In this paper we introduced DÏoT: a self-learning system for detecting compromised devices in IoT networks. Our solution relies on novel automated techniques for *device-type-specific anomaly detection*. DÏoT does *not require any human intervention or labeled data* to operate. It learns anomaly detection models autonomously, using unlabeled crowdsourced data captured in client IoT networks. We demonstrated the efficacy of anomaly detection in detecting a large set of malicious behavior from devices infected by the Mirai malware. DÏoT detected 95.6% of attacks in 257 milliseconds on average and without raising any false alarm when evaluated in a real-world deployment.

Acknowledgments: This work was supported in part by the Academy of Finland (SELIoT project - grant 309994), the German Research Foundation (DFG) within CRC 1119 CROSSING (S2 and P3) and the Intel Collaborative Institute for Collaborative Autonomous and Resilient Systems (ICRICARS). We would also like to thank Cisco Systems, Inc. for their support of this work.

References

[1] M. Antonakakis, T. April, M. Bailey, M. Bernhard, E. Bursztein, J. Cochran, Z. Durumeric, J. A. Halderman, L. Invernizzi, M. Kallitsis, D. Kumar, C. Lever, Z. Ma, J. Mason, D. Menscher, C. Seaman, N. Sullivan, K. Thomas, and Y. Zhou, "Understanding the mirai botnet," in *26th USENIX Security Symposium (USENIX Security 17)*. Vancouver, BC: USENIX Association, 2017, pp. 1093–1110.

[2] S. Edwards and I. Profetis, "Hajime: Analysis of a decentralized internet worm for IoT devices," Rapidity Networks, Tech. Rep., 2016.

[3] C. Kolias, G. Kambourakis, A. Stavrou, and J. Voas, "DDoS in the IoT: Mirai and other botnets," *Computer*, vol. 50, no. 7, pp. 80–84, 2017.

[4] Radware, "BrickerBot results in PDoS attack," https://security.radware.com/ddos-threats-attacks/brickerbot-pdos-permanent-denial-of-service/, 2017, [Online; accessed 8-April-2019].

[5] N. Hadar, S. Siboni, and Y. Elovici, "A lightweight vulnerability mitigation framework for iot devices," in *Proceedings of the 2017 Workshop on Internet of Things Security and Privacy*, ser. IoTS&P '17. New York, NY, USA: ACM, 2017, pp. 71–75. [Online]. Available: http://doi.acm.org/10.1145/3139937.3139944

[6] R. Doshi, N. Apthorpe, and N. Feamster, "Machine learning ddos detection for consumer internet of things devices," *CoRR*, vol. abs/1804.04159, 2018. [Online]. Available: http://arxiv.org/abs/1804.04159

Authorized licensed use limited to: Universidad Nacional de Ingeniería. Downloaded on October 20,2026 at 06:07:28 UTC from IEEE Xplore. Restrictions apply.

[7] C. Krügel, T. Toth, and E. Kirda, "Service specific anomaly detection for network intrusion detection," in *Proceedings of the 2002 ACM symposium on Applied computing*. ACM, 2002, pp. 201–208.

[8] L. Portnoy, E. Eskin, and S. Stolfo, "Intrusion detection with unlabeled data using clustering," in *In Proceedings of ACM CSS Workshop on Data Mining Applied to Security*, 2001.

[9] S. Rajasegarar, C. Leckie, and M. Palaniswami, "Hyperspherical cluster based distributed anomaly detection in wireless sensor networks," *Journal of Parallel and Distributed Computing*, vol. 74, no. 1, pp. 1833–1847, 2014.

[10] V. Costan and S. Devadas, "Intel SGX explained." *IACR Cryptology ePrint Archive*, vol. 2016, p. 86, 2016.

[11] G. Dessouky, S. Zeitouni, T. Nyman, A. Paverd, L. Davi, P. Koeberl, N. Asokan, and A.-R. Sadeghi, "LO-FAT: Low-overhead control flow attestation in hardware," in *Design Automation Conference (DAC), 2017 54th ACM/EDAC/IEEE*. IEEE, 2017, pp. 1–6.

[12] X. Feng, Q. Li, H. Wang, and L. Sun, "Acquisitional rule-based engine for discovering internet-of-things devices," in *27th USENIX Security Symposium (USENIX Security 18)*, 2018, pp. 327–341.

[13] M. Miettinen, S. Marchal, I. Hafeez, N. Asokan, A. Sadeghi, and S. Tarkoma, "IoT Sentinel: Automated Device-Type Identification for Security Enforcement in IoT," in *Proc. 37th IEEE International Conference on Distributed Computing Systems (ICDCS 2017)*, Jun. 2017.

[14] S. Marchal, M. Miettinen, T. D. Nguyen, A. Sadeghi, and N. Asokan, "Audi: Towards autonomous iot device-type identification using periodic communication," *IEEE Journal on Selected Areas in Communications* , pp. 1–1, 2019.

[15] E. Lear, R. Droms, and D. Romascanu, "Manufacturer usage description specification," https://tools.ietf.org/html/draft-ietf-opsawg-mud-25, IETF Network Working Group, [Online; accessed 8-April-2019].

[16] keras.io, "Gated recurrent unit," https://keras.io/layers/recurrent/, 2014, [Online; accessed 8-April-2019].

[17] J. Chung, Ç. Gülçehre, K. Cho, and Y. Bengio, "Empirical evaluation of gated recurrent neural networks on sequence modeling," *CoRR*, vol. abs/1412.3555, 2014, http://arxiv.org/abs/1412.3555.

[18] C.-Y. Wu, A. Ahmed, A. Beutel, A. J. Smola, and H. Jing, "Recurrent recommender networks," in *Proceedings of the Tenth ACM International Conference on Web Search and Data Mining*, ser. WSDM '17. New York, NY, USA: ACM, 2017, pp. 495–503. [Online]. Available: http://doi.acm.org/10.1145/3018661.3018689

[19] J. Konecný, B. McMahan, F. X. Yu, P. Richtárik, A. T. Suresh, and D. Bacon, "Federated learning: Strategies for improving communication efficiency," *CoRR*, vol. abs/1610.05492, 2016.

[20] V. Smith, C.-K. Chiang, M. Sanjabi, and A. S. Talwalkar, "Federated multi-task learning," in *Advances in Neural Information Processing Systems*, I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, Eds., 2017, pp. 4427–4437.

[21] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proceedings of the 20 th International Conference on Artificial Intelligence and Statistics*, 2017, pp. 1273–1282.

[22] "A micro web framework written in python," http://flask.pocoo.org, [Online; accessed 8-April-2019].

[23] "Flask socketio," https://flask-socketio.readthedocs.io/en/latest/, [Online; accessed 8-April-2019].

[24] "Flask socketio client," https://github.com/socketio/socket.io-client, [Online; accessed 8-April-2019].

[25] "gevent asynchronous framework," https://github.com/gevent/gevent, [Online; accessed 8-April-2019].

[26] "Keras deep learning library," https://faroit.github.io/keras-docs/2.0.2/, [Online; accessed 8-April-2019].

[27] T. Yeh, D. Chiu, and K. Lu, "Persirai: New internet of things (IoT) botnet targets IP cameras," https://blog.trendmicro.com/trendlabs-security-intelligence/persirai-new-internet-things-iot-botnet-targets-ip-cameras/, TrendMicro, [Online; accessed 8-April-2019].

[28] Y. M. P. Pa, S. Suzuki, K. Yoshioka, T. Matsumoto, T. Kasama, and C. Rossow, "IoTPOT: A novel honeypot for revealing current IoT threats," *Journal of Information Processing*, vol. 24, no. 3, pp. 522–533, 2016.

[29] T. D. Nguyen, S. Marchal, M. Miettinen, H. Fereidooni, N. Asokan, and A. Sadeghi, "Dïot: A federated self-learning anomaly detection system for iot," *arXiv:1804.07474*, 2018.

[30] J. Gamblin, "Mirai source code," https://github.com/jgamblin/Mirai-Source-Code, Jul. 2017, [Online; accessed 8-April-2019].

[31] N. Carlini and D. Wagner, "Towards evaluating the robustness of neural networks," in *Security and Privacy (SP), 2017 IEEE Symposium on*. IEEE, 2017, pp. 39–57.

[32] A. Kurakin, I. Goodfellow, and S. Bengio, "Adversarial examples in the physical world," *arXiv preprint arXiv:1607.02533*, 2016.

[33] I. Evtimov, K. Eykholt, E. Fernandes, T. Kohno, B. Li, A. Prakash, A. Rahmati, and D. Song, "Robust physical-world attacks on machine learning models," *arXiv preprint arXiv:1707.08945*, 2017.

[34] T. Vaidya, Y. Zhang, M. Sherr, and C. Shields, "Cocaine noodles: Exploiting the gap between human and machine speech recognition," in *9th USENIX Workshop on Offensive Technologies (WOOT 15)*. Washington, D.C.: USENIX Association, 2015. [Online]. Available: https://www.usenix.org/conference/woot15/workshop-program/presentation/vaidya

[35] G. Zhang, C. Yan, X. Ji, T. Zhang, T. Zhang, and W. Xu, "Dolphinattack: Inaudible voice commands," in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*. ACM, 2017, pp. 103–117.

[36] B. Biggio, B. Nelson, and P. Laskov, "Poisoning attacks against support vector machines," in *Proceedings of the 29th International Coference on International Conference on Machine Learning*. Omnipress, 2012, pp. 1467–1474.

[37] B. I. Rubinstein, B. Nelson, L. Huang, A. D. Joseph, S.-h. Lau, S. Rao, N. Taft, and J. Tygar, "Antidote: understanding and defending against poisoning of anomaly detectors," in *Proceedings of the 9th ACM SIGCOMM conference on Internet measurement*. ACM, 2009, pp. 1–14.

[38] S. Shen, S. Tople, and P. Saxena, "Auror: Defending against poisoning attacks in collaborative deep learning systems," in *Proceedings of the 32Nd Annual Conference on Computer Security Applications*, ser. ACSAC '16. ACM, 2016, pp. 508–519.

[39] Y. J. Jia, Q. A. Chen, S. Wang, A. Rahmati, E. Fernandes, Z. M. Mao, and A. Prakash, "ContexIoT: Towards providing contextual integrity to appified IoT platforms," in *24th Annual Network & Distributed System Security Symposium (NDSS)*, feb 2017.

[40] S. Raza, L. Wallgren, and T. Voigt, "Svelte: Real-time intrusion detection in the internet of things," *Ad hoc networks*, vol. 11, no. 8, pp. 2661–2674, 2013.

[41] W. Jardine, S. Frey, B. Green, and A. Rashid, "Senami: Selective non-invasive active monitoring for ics intrusion detection," in *Proceedings of the 2Nd ACM Workshop on Cyber-Physical Systems Security and Privacy*, ser. CPS-SPC '16, 2016, pp. 23–34.

[42] A. Kleinmann and A. Wool, "Automatic construction of statechart-based anomaly detection models for multi-threaded scada via spectral analysis," in *Proceedings of the 2Nd ACM Workshop on Cyber-Physical Systems Security and Privacy*, ser. CPS-SPC '16. ACM, 2016, pp. 1–12.

[43] R. Sekar, A. Gupta, J. Frullo, T. Shanbhag, A. Tiwari, H. Yang, and S. Zhou, "Specification-based anomaly detection: a new approach for detecting network intrusions," in *Proceedings of the 9th ACM conference on Computer and communications security*. ACM, 2002, pp. 265–274.

[44] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," in *Security and Privacy (SP), 2010 IEEE Symposium on*. IEEE, 2010, pp. 305–316.

[45] P. Malhotra, L. Vig, G. Shroff, and P. Agarwal, "Long short term memory networks for anomaly detection in time series," in *Proceedings*. Presses universitaires de Louvain, 2015, p. 89.

[46] A. Nanduri and L. Sherry, "Anomaly detection in aircraft data using recurrent neural networks (rnn)," in *Integrated Communications Navigation and Surveillance (ICNS), 2016*. IEEE, 2016, pp. 5C2–1.

[47] M. Du, F. Li, G. Zheng, and V. Srikumar, "Deeplog: Anomaly detection and diagnosis from system logs through deep learning," in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '17. ACM, 2017, pp. 1285–1298, http://doi.acm.org/10.1145/3133956.3134015.

[48] A. Oprea, Z. Li, T.-F. Yen, S. H. Chin, and S. Alrwais, "Detection of early-stage enterprise infection by mining large-scale log data," in *Dependable Systems and Networks (DSN), 2015 45th Annual IEEE/IFIP International Conference on*. IEEE, 2015, pp. 45–56.

Authorized licensed use limited to: Universidad Nacional de Ingeniería. Downloaded on October 20,2026 at 06:07:28 UTC from IEEE Xplore. Restrictions apply.